

Vision Transformers

Foundations, Variants, and Practical Implementation

Prof. Hamid D. Taghirad · Mohammad Javad Ahmadi

K. N. Toosi University of Technology – Faculties of Electrical & Computer Engineering aras.kntu.ac.ir/neura · 2026

ARAS × Neura Academy

Five movements, one architecture

I

Foundations & core mechanics

Tokenizing images, embeddings, positional encoding, self-attention.

II

Architectures & variants

ViT, DeiT, Swin, MViT, MAE, DINOv2, CLIP, video ViTs.

III

Robotics & vision applications

Dense prediction, open-vocab perception, VLA policies.

IV

Programming & implementation

PyTorch, timm, HF; fine-tuning, AMP, profiling, LoRA.

V

A practical decision guide

Choosing a backbone and a fine-tuning regime.

Learning objectives & prerequisites

ON COMPLETION YOU WILL BE ABLE TO –

- 01 Derive the token representation of an image and explain why attention scales quadratically in token count.
- 02 Locate any modern backbone — hierarchical, self-supervised, vision–language, spatio-temporal — within a coherent taxonomy.
- 03 Select and adapt a foundation backbone for a robotic perception task under a fixed compute budget.
- 04 Fine-tune a ViT with mixed precision, profile it, and apply parameter-efficient adaptation.

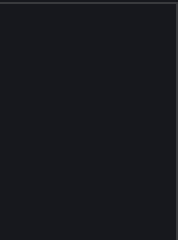
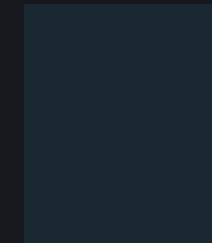
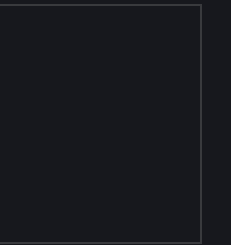
ASSUMED BACKGROUND

- ▶ Backpropagation & the optimization of deep networks
- ▶ Convolutional architectures & the notion of a receptive field
- ▶ Working fluency in Python & PyTorch tensors
- ▶ Elementary linear algebra: matrix products, softmax

PART I

Foundations & core mechanics

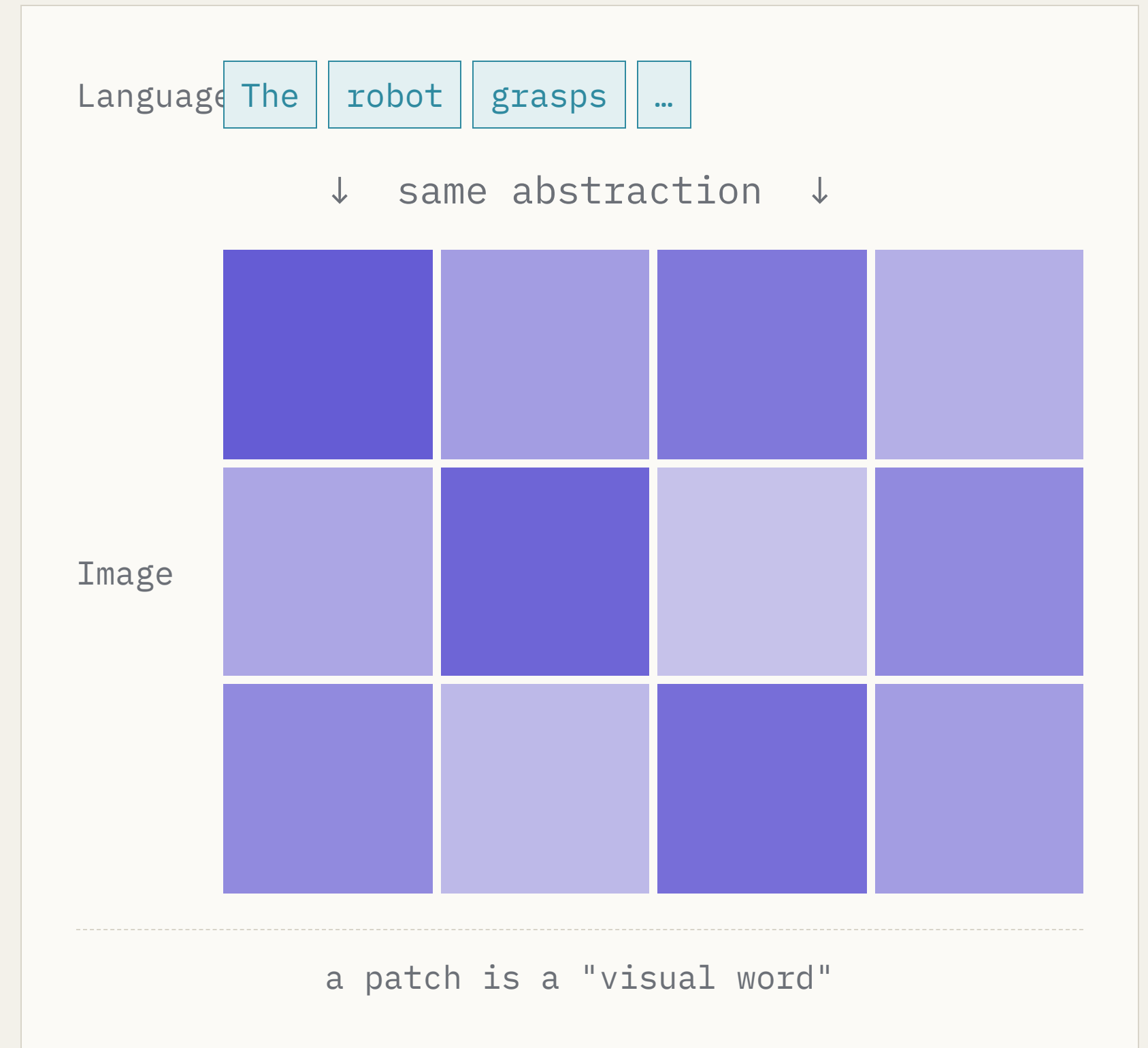
From a grid of pixels to a sequence of tokens — and the attention operator that binds them into a global representation.



From sequences to pixels: the NLP-to-vision transfer

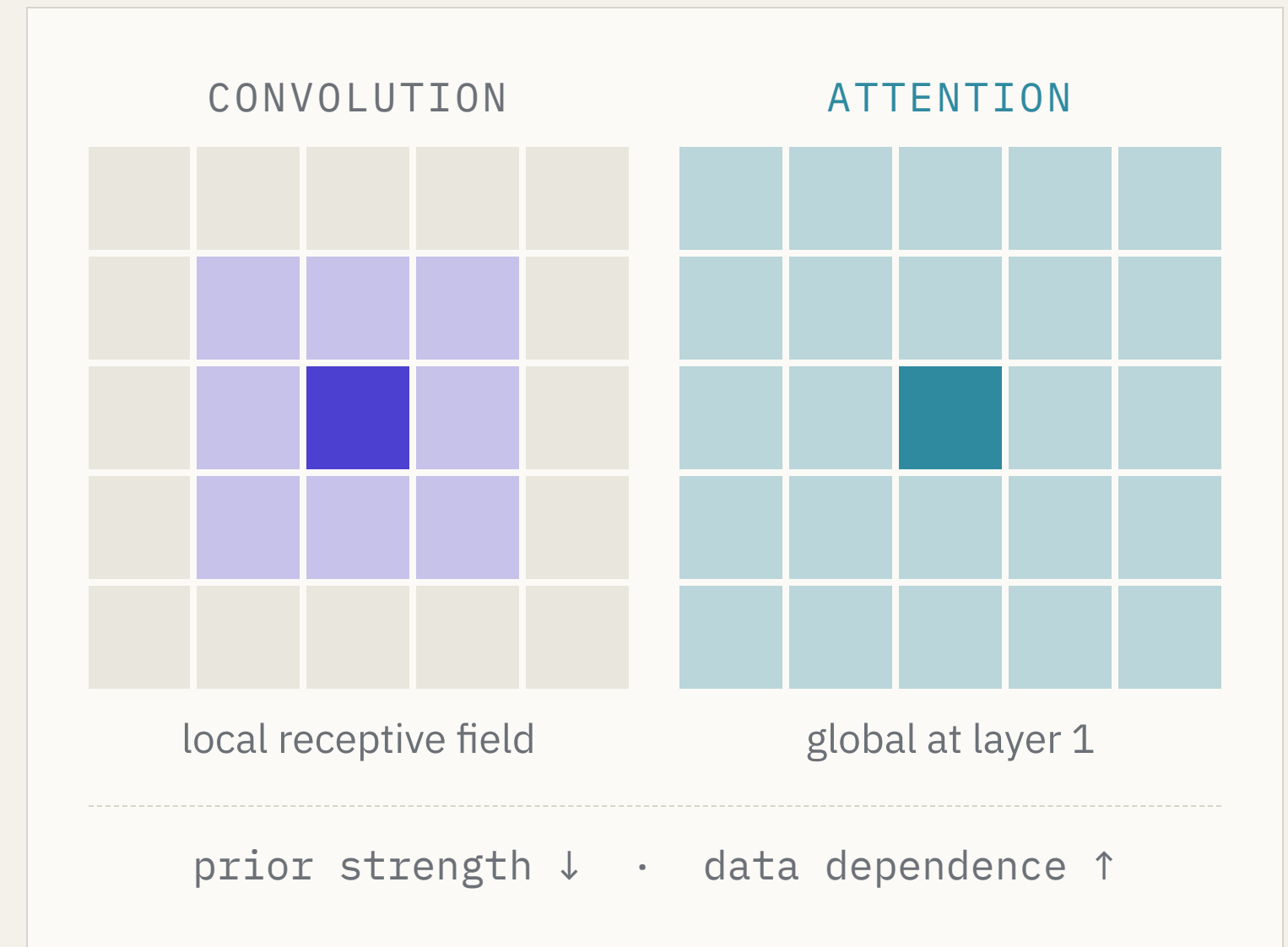
The transformer dispensed with recurrence and convolution alike, modeling a sequence purely as **tokens related by attention**. Its dominance in language posed an obvious question for vision.

- **The tokenization problem.** A word is a discrete unit; a pixel is not. We need a visual primitive coarse enough to be tractable, fine enough to be expressive.
- **ViT's answer.** Partition the image into a grid of non-overlapping patches; treat each as a token. Vision becomes *set-to-set* sequence modeling.
- **What this unlocks.** The full language toolchain transfers: scaling laws, masked pretraining, and native multimodal fusion with text.

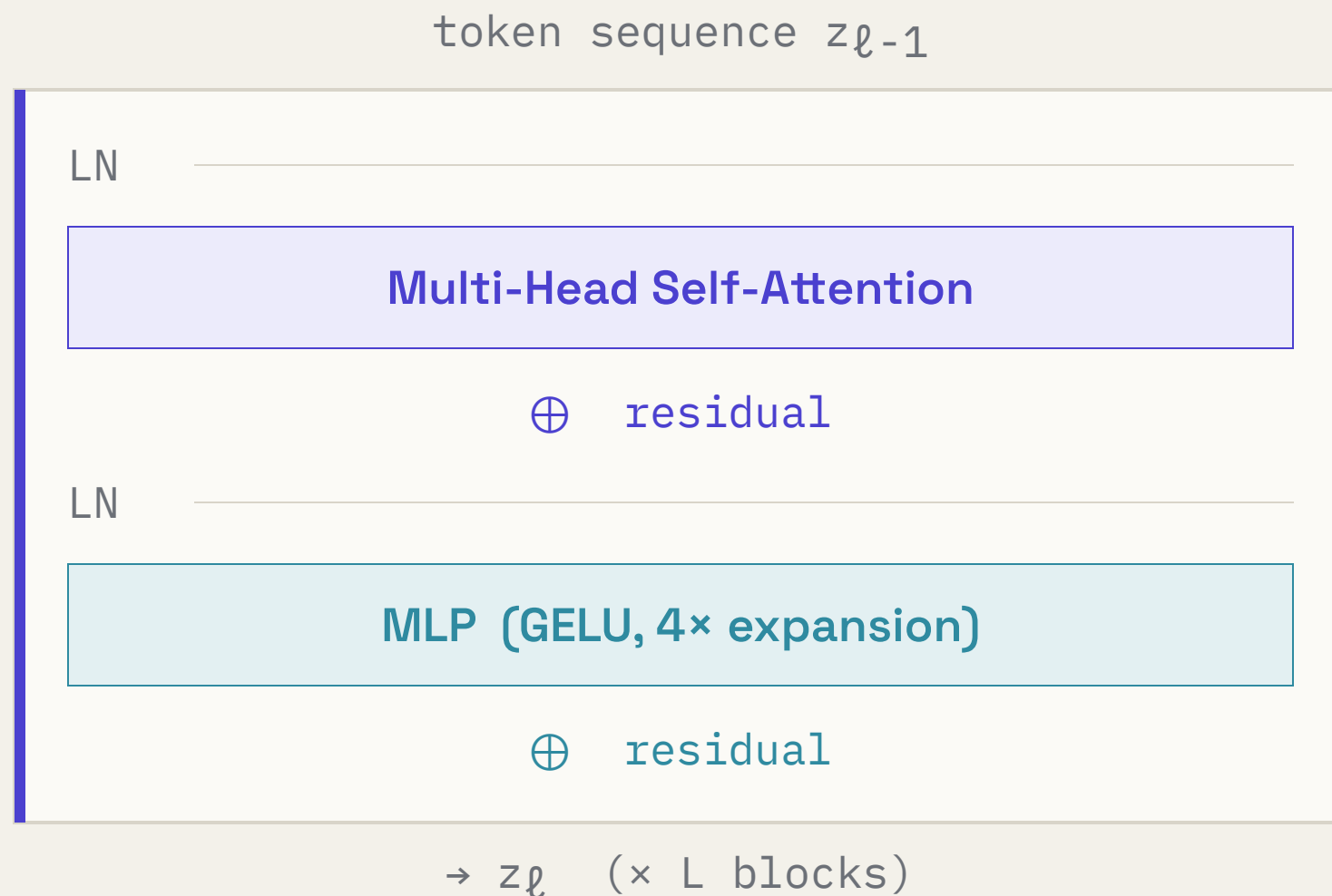


Limits of the convolutional inductive bias

- 01 **Locality & weight sharing** are strong priors — superb sample efficiency on natural images, but they presuppose that relevant structure is spatially local.
- 02 **Receptive field grows slowly.** Global context emerges only after many layers of stacking; modeling distant relations is structurally awkward.
- 03 **Translation equivariance is baked in** — desirable, yet inflexible. The prior cannot be relaxed where the task would benefit.
- 04 **Attention trades prior for flexibility.** Global mixing at every layer — but with weak priors, performance becomes contingent on data scale or distillation.



The transformer encoder, recapped



- **Encoder only.** No decoder and no causal mask — every token sees every other, bidirectionally.
- **Two residual sub-layers.** Attention mixes tokens; the MLP refines each token independently. Residuals make the block a refinement step.
- **Pre-norm placement.** LayerNorm before each sub-layer keeps gradients well-scaled — essential for training deep stacks.
- **Where the capacity lives.** The 4× MLP holds most parameters and supplies per-token nonlinear expressivity.

Self-attention: query-key-value mechanics

Q Query — what this token is looking for. A learned projection of the token.

K Key — what each token advertises. Query·key measures relevance.

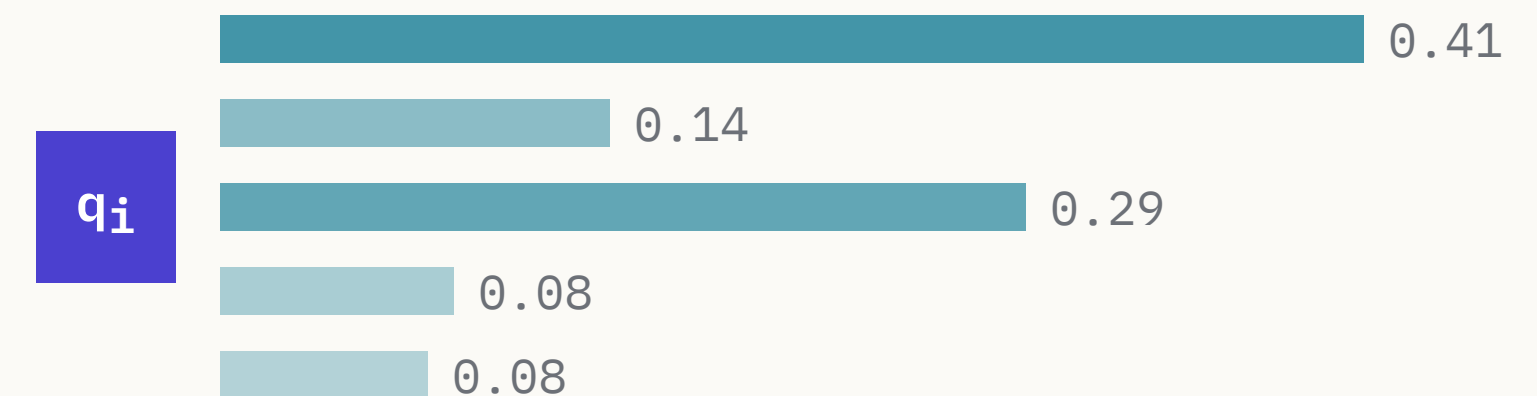
V Value — the content a token contributes once attended to.

SCALED DOT-PRODUCT ATTENTION

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

$\sqrt{d_k}$ rescales logits into softmax's high-gradient regime.

ONE QUERY ATTENDS TO ALL KEYS

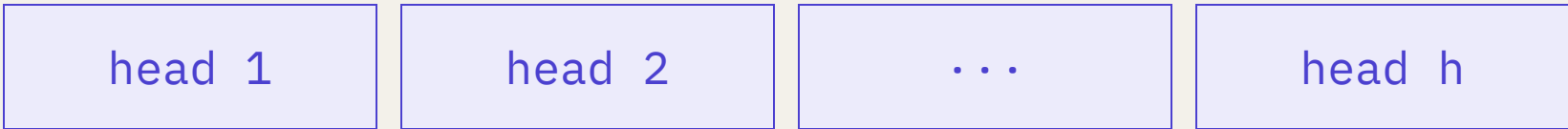


$$\text{attention weights} = \text{softmax over keys} \rightarrow \sum w_j \cdot v_j$$

The output token is a **convex blend of values**, routed by content similarity — differentiable, permutation-equivariant soft retrieval.

Multi-head attention & the quadratic wall

Multi-head. Project into h parallel subspaces, attend independently, then concatenate and project. Heads specialize — local texture, long-range structure — enriching the relational vocabulary at fixed asymptotic cost.



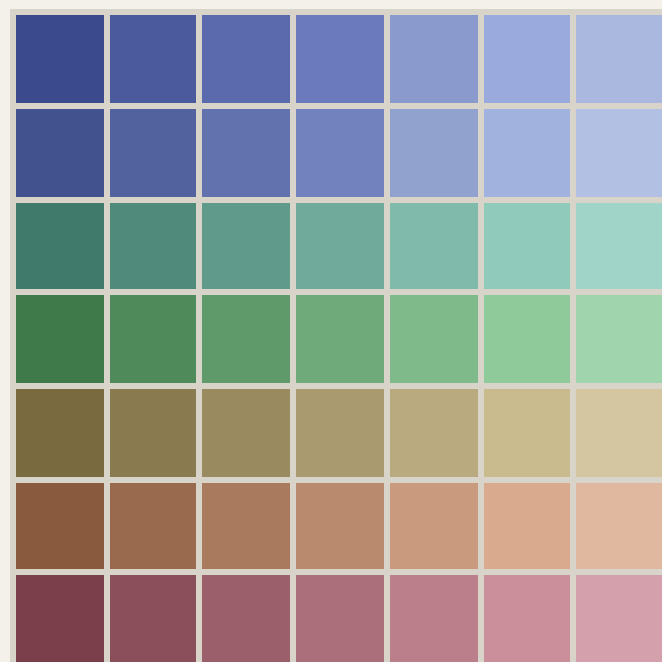
COST OF SELF-ATTENTION

$\mathcal{O}(N^2 \cdot d)$

N = number of tokens · the $N \times N$ score matrix dominates memory.

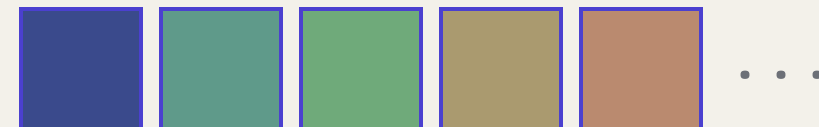
TOKENS GROW WITH RESOLUTION		
input (16×16 patch)	tokens N	N^2 scores
224 × 224	196	38 K
384 × 384	576	332 K
768 × 768	2304	5.3 M
1024 × 1024	4096	16.8 M
Dense prediction and high-resolution inputs hit this wall — the central pressure shaping Part II.		

Patch extraction & tokenization



224 × 224 × 3

tile & flatten →



↓ each patch flattened



patch size

16 × 16

grid

14 × 14

sequence length N

196 tokens

Linear patch embedding & the [CLS] token

- **Shared linear projection.** One learned matrix maps every flattened patch $\mathbb{R}^{768} \rightarrow \mathbb{R}^D$, the model width.
- **It is just a convolution.** A `Conv2d(3, D, k=16, s=16)` performs tiling and projection in one op.
- **The [CLS] token.** A learnable, content-free vector prepended to the sequence; its final state is the pooled image representation.
- **Modern alternative.** Global average pooling over patch tokens often matches or exceeds [CLS] — standard in DINOv2-era backbones.

INPUT SEQUENCE TO THE ENCODER



▲ read out for classification

$$z_0 = [x_{cls} ; x^1_{pE} ; \dots ; x^N_{pE}] + E_{pos}$$

Sequence length becomes $N + 1$; width D is constant through every block.

Positional encoding for 2D structure

Why it is needed — self-attention is permutation-equivariant. Shuffle the tokens and the output shuffles identically; the model is blind to location unless position is injected.

LEARNABLE 1D

Original ViT

One trainable vector per position, added to tokens.
Simple; ties the model to a fixed grid.

2D-AWARE

Factorized row/col

Separate embeddings for x and y coordinates respect the grid's geometry explicitly.

RELATIVE BIAS

Swin-style

Encodes pairwise *offsets*, not absolute coordinates — transfers across resolutions gracefully.

INTERPOLATED / RoPE

Resolution transfer

Bicubic-interpolate the grid, or use 2D rotary phases, to fine-tune at new input sizes.

The Vision Transformer, end to end



Invariant – width D constant throughout

Scaling knobs – depth L, width D, patch size

Part II – perturbs one stage of this pipeline

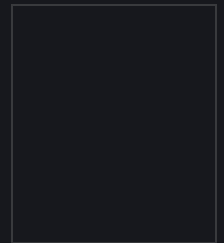
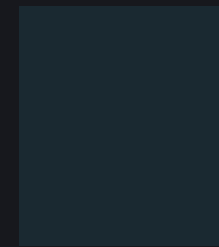
ViT vs. CNN: a structured comparison

PROPERTY	Convolutional net	Vision Transformer
Inductive bias	Strong — locality, equivariance	Weak — learned from data
Receptive field	Grows with depth	Global at every layer
Data efficiency	High — strong in small-data	Low — needs scale or distillation
Compute scaling	Linear in pixels	Quadratic in tokens
Long-range modeling	Indirect, deep	Native, single hop
Best regime	Limited data, edge compute	Large-scale pretraining & transfer

PART II

Architectures & variants

Reintroducing hierarchy, locality, and self-supervision — without surrendering attention's global reach.



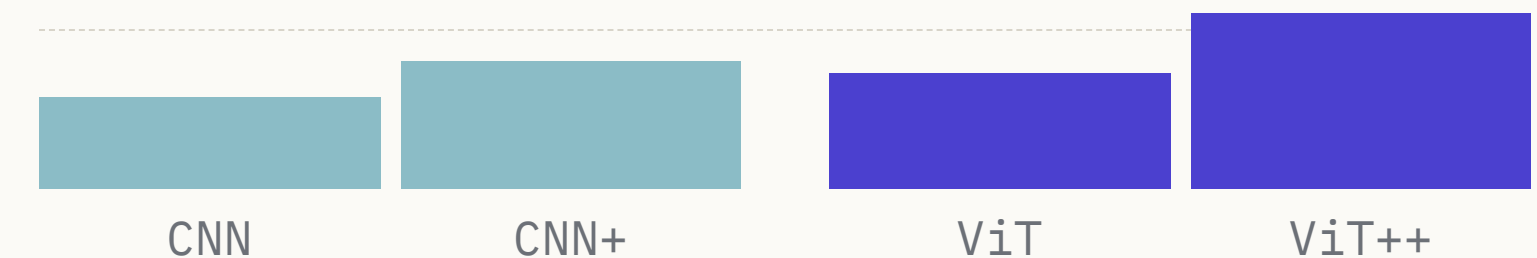
ViT: original formulation & scaling behavior

Dosovitskiy et al. (2021) argued that convolutional priors are **dispensable at scale**: a near-vanilla encoder over image patches matches the best CNNs — provided pretraining data is large enough.

- **Data threshold.** On ImageNet-1k alone, ViT trails ResNets; pretrained on JFT-300M it overtakes them and keeps climbing.
- **Scaling mindset.** Accuracy is a smooth function of data, parameters, and compute — architecture search becomes resource allocation.
- **Notation.** ViT-B/16 = Base variant, 16-px patches. Smaller patch → more tokens → higher cost and accuracy.

CANONICAL CONFIGURATIONS

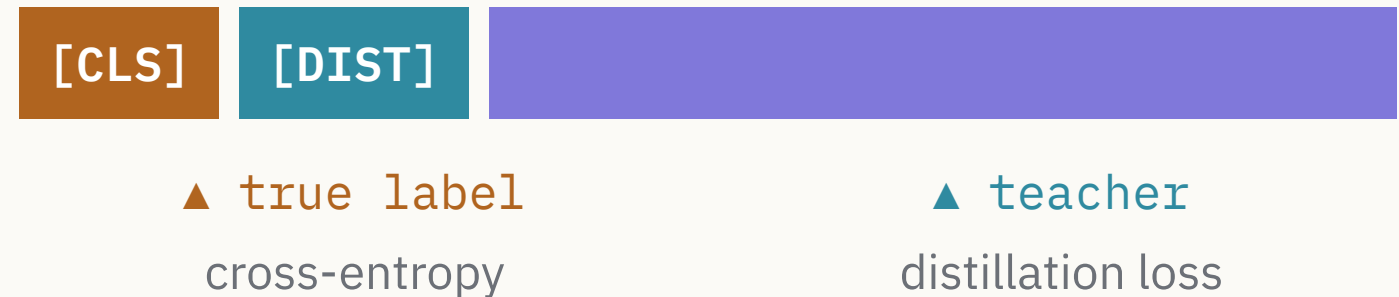
model	layers	width	params
ViT-B	12	768	86 M
ViT-L	24	1024	307 M
ViT-H	32	1280	632 M



Data efficiency & DeiT: the distillation token

- **No JFT required.** DeiT reaches competitive ImageNet accuracy on ImageNet-1k alone, on a single node in days — recipe, not scale.
- **Heavy augmentation.** RandAugment, Mixup, CutMix, stochastic depth, repeated augmentation — together they substitute for missing data.
- **The distillation token.** A second special token learns to match a teacher's prediction — typically a strong CNN, transferring its inductive bias.
- **The lesson.** Priors can be injected through the *optimization target*, not only the architecture.

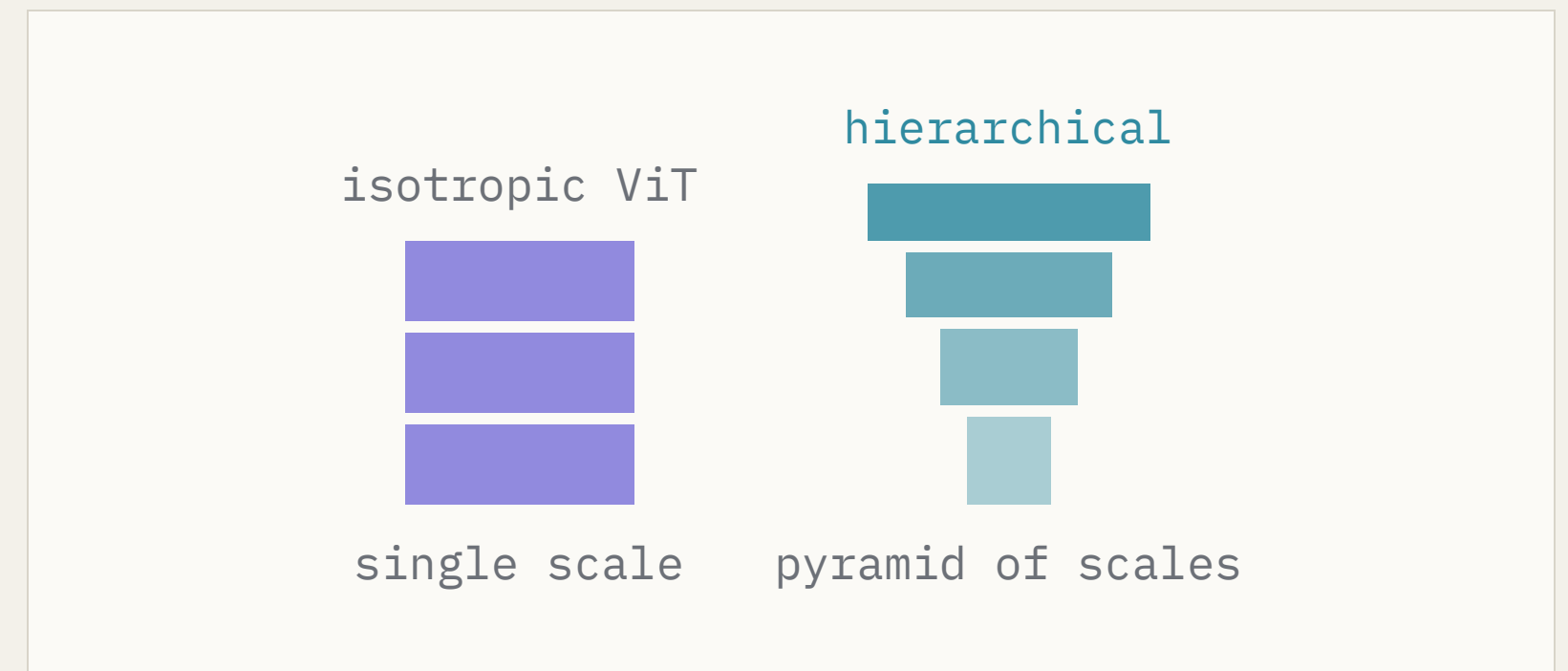
TWO READ-OUT HEADS



At inference the two heads are averaged. The CNN teacher's locality bias is absorbed by the student transformer.

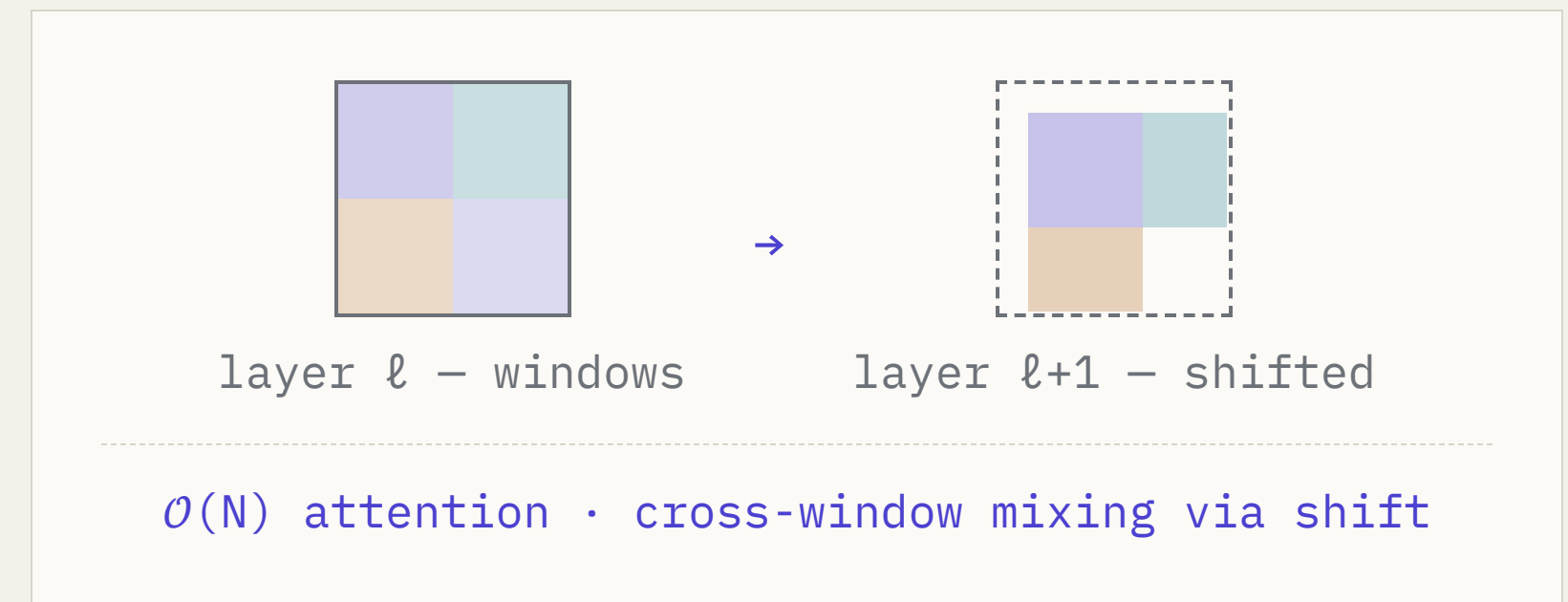
The case for hierarchy in dense prediction

- **Plain ViT is isotropic.** One resolution, one token count throughout — ideal for classification, ill-suited to pixel-level tasks.
- **Dense prediction needs scale.** Detection and segmentation require multi-scale features and high-resolution maps — quadratic cost forbids both in a flat ViT.
- **Port the feature pyramid.** Progressively merge tokens — halve spatial resolution, widen channels — recreating the CNN pyramid inside a transformer.
- **Result.** Drop-in backbones for FPN-style detectors and segmentors — Swin, PVT, MViT.



Swin Transformer: shifted-window attention

- **Windowed attention.** Attend only within fixed local windows (e.g. 7×7 patches) $\rightarrow$ cost *linear* in image area, not quadratic.
- **Shifted windows.** Alternate blocks displace the grid by half a window, so partitions straddle prior boundaries — information propagates globally in a few layers.
- **Patch merging.** Between stages, merge 2×2 token neighborhoods — the pyramid that yields multi-scale features.
- **Impact.** A relative-position-biased, general-purpose backbone — for years the default for detection and segmentation.



PVT & Multiscale ViT: two routes to the pyramid

PVT · SPATIAL-REDUCTION ATTENTION

Keep global reach, shrink the keys

- ▶ Downsample K and V before the dot product → smaller $N \times N$ matrix.
- ▶ Every query still attends to a coarsened **global** field.
- ▶ Trades *context resolution* for globality — the mirror of Swin.

MViT · POOLING ATTENTION

Pool Q, K, V inside attention

- ▶ Pooling operators build a feature hierarchy within the attention op.
- ▶ Designed for **video** — a hierarchy across space *and* time.
- ▶ Channels widen as spatio-temporal resolution contracts.

Meta-point — "hierarchical ViT" is a family, not a trick: Swin localizes, PVT coarsens context, MViT pools — each a different incision into the quadratic cost.

Efficient attention: linear & sparse families

FAMILY	MECHANISM	EXAMPLES	COST
Local / windowed	Restrict attention to local windows	Swin, NAT	$\mathcal{O}(N)$
Low-rank / linear	Approximate the softmax kernel	Linformer, Performer	$\mathcal{O}(N)$
Sparse / factorized	Attend along structured subsets	Axial, BigBird	$\mathcal{O}(N\sqrt{N})$
IO-aware exact	Same math — never materialize $N\times N$ in HBM	FlashAttention 2/3	exact

Approximate methods change the *operator*. **FlashAttention** changes only the *implementation* — usually the first thing to reach for.

Hybrid designs: convolution meets attention

- **Convolutional stems.** Early conv layers supply cheap low-level features and inject locality — stabilizing ViT optimization (CvT, LeViT).
- **Conv token embedding.** Overlapping convolutional patchification preserves local continuity that hard tiling discards.
- **ConvNeXt — the counterpoint.** A *pure CNN* modernized with transformer design choices (large kernels, LayerNorm, inverted bottleneck) matches Swin.
- **The lesson.** Much of the ViT edge was recipe and macro-design — not attention alone. The frontier is convergent.

CONVERGENT DESIGN

CNN →

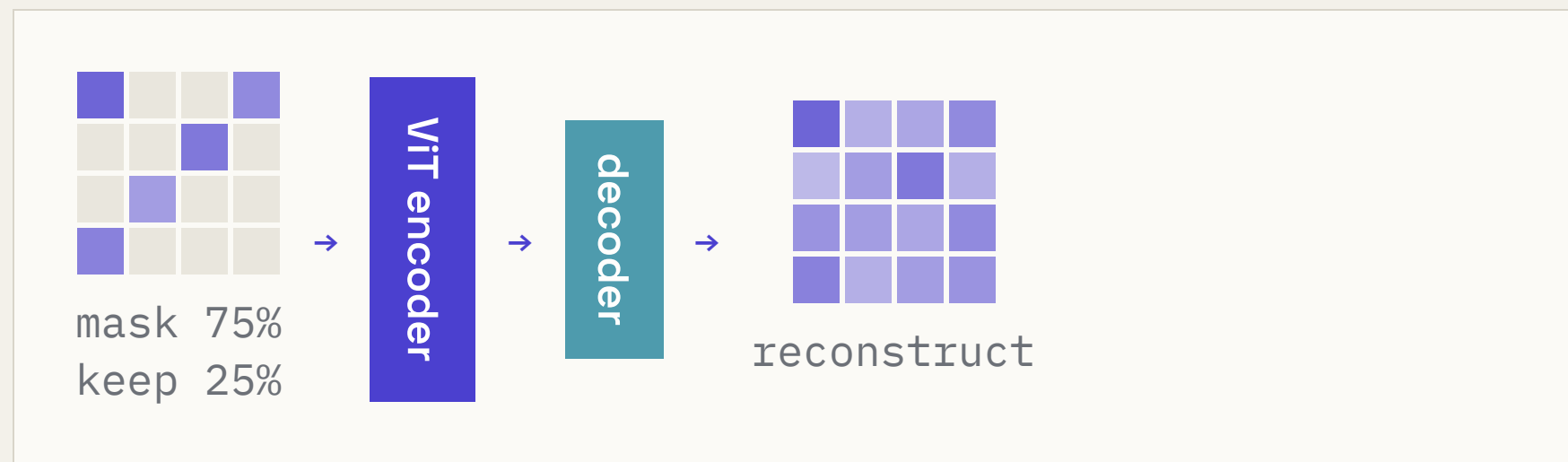
ViT →



large-kernel · LayerNorm · GELU · conv-stem hybrids

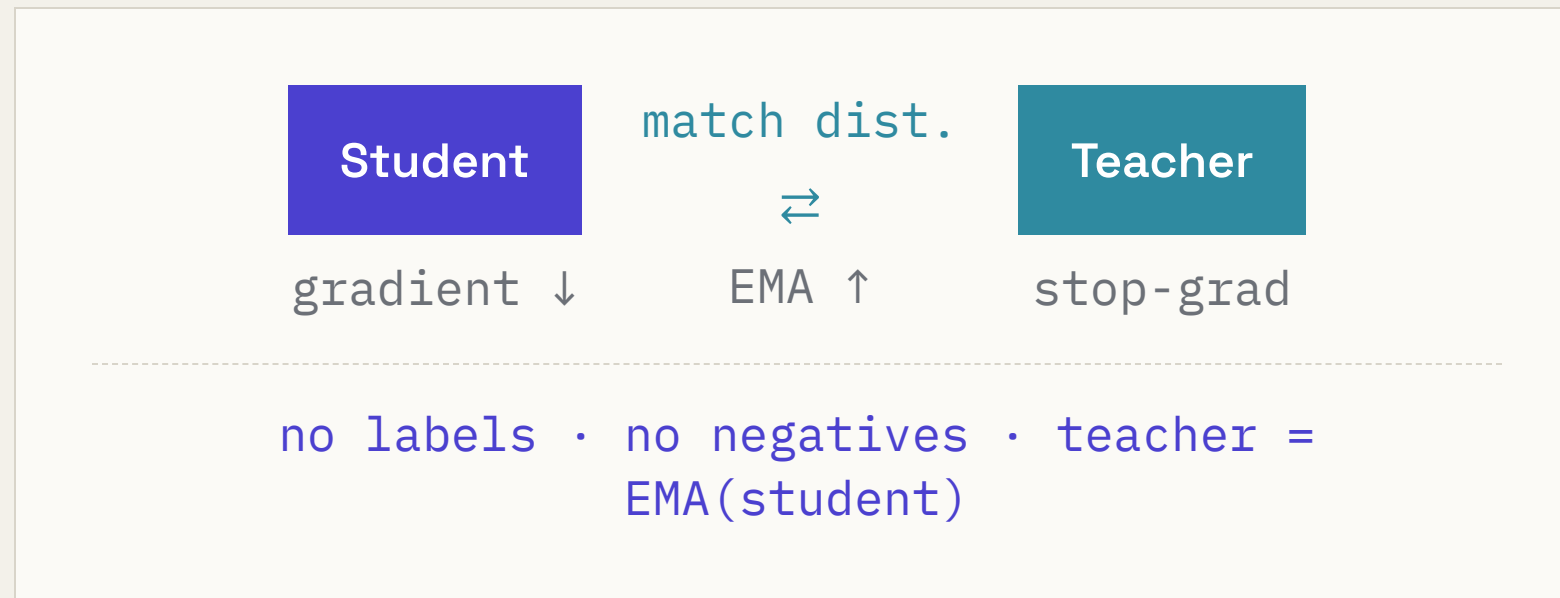
Both lineages absorb each other's macro-design — the best 2024-era backbones are explicitly hybrid.

Masked image modeling: MAE & BEiT



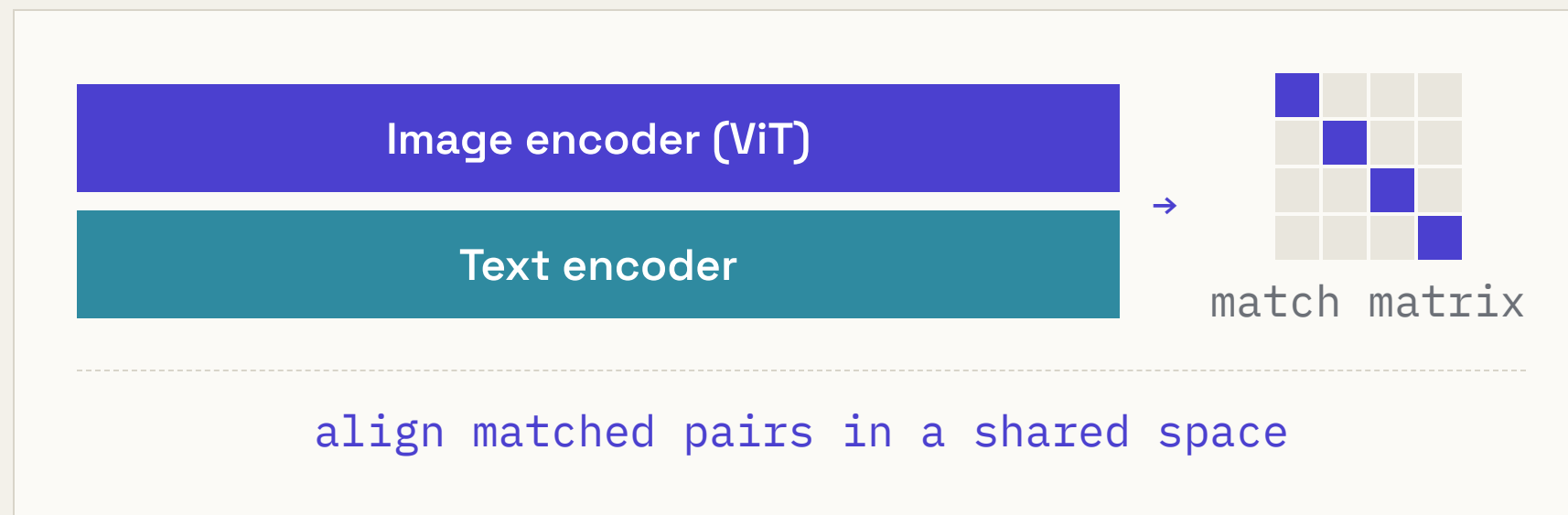
- **MAE.** Encode only visible patches; a light decoder reconstructs masked pixels. The 75% mask forces semantics over texture.
- **Cheap by design.** The encoder sees a quarter of the tokens — pretraining cost drops sharply.
- **BEiT.** Predict discrete visual tokens from a learned dVAE — masked modeling as classification, not regression.
- **Payoff.** Label-free encoders that fine-tune to state of the art — the BERT moment for vision.

Self-distillation: DINO & DINOv2



- **Emergent segmentation.** Attention maps localize objects with no segmentation supervision; frozen-feature k-NN is competitive.
- **DINOv2 scaled it.** Large curated data + register tokens (fixing attention artifacts) yield general-purpose frozen features.
- **Transfers out of the box.** Depth, segmentation, retrieval, correspondence — often with a linear probe, no fine-tuning.
- ★ **For robotics.** A frozen, all-purpose visual feature extractor — the default perception backbone in many modern stacks.

Vision-language pretraining: CLIP & SigLIP

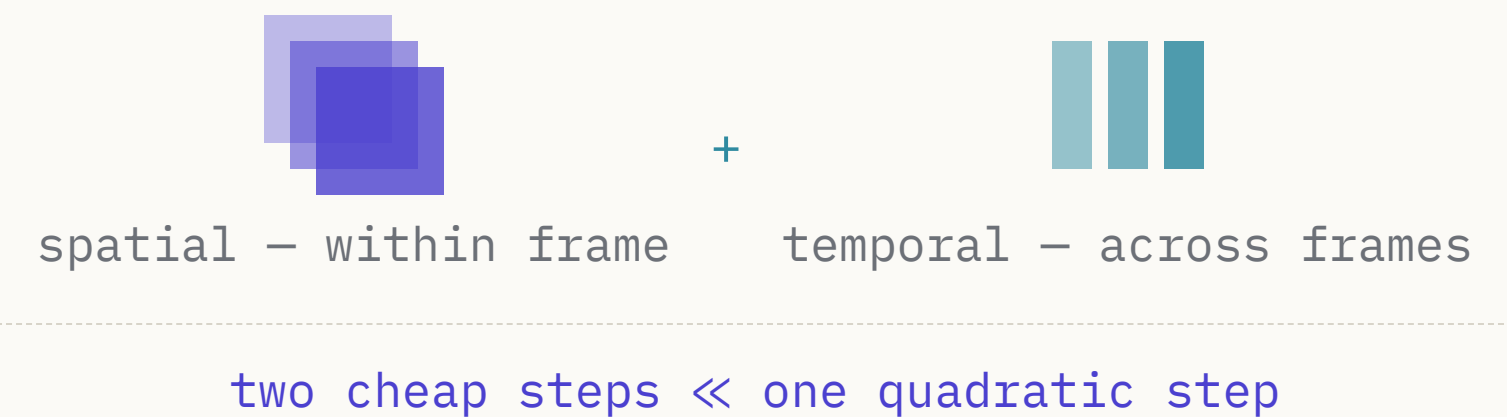


- **Contrastive alignment.** Pull matched image–caption pairs together, push mismatches apart over a web-scale corpus.
- **Zero-shot.** Classify against arbitrary text prompts at inference — never trained on those classes.
- **SigLIP.** A pairwise *sigmoid* loss decouples the objective from batch size — more stable, better scaling.
- ★ **For robots.** Language becomes a queryable interface to perception — open-vocabulary, not a fixed label set.

Spatio-temporal ViTs: ViViT & TimeSformer

- **The problem.** Joint space-time attention scales with $(\text{frames} \times \text{patches})^2$ — intractable for real clips.
- **Tubelet embedding.** Patchify in 3D — each token is a small space-time volume spanning several frames.
- **Divided attention.** TimeSformer alternates spatial (within frame) and temporal (across frames) attention; ViViT factorizes the encoder.
- ★ **For robotics.** Action recognition, video prediction, and temporally-grounded policies.

FACTORIZED SPACE-TIME ATTENTION



The architecture landscape: a comparative synthesis

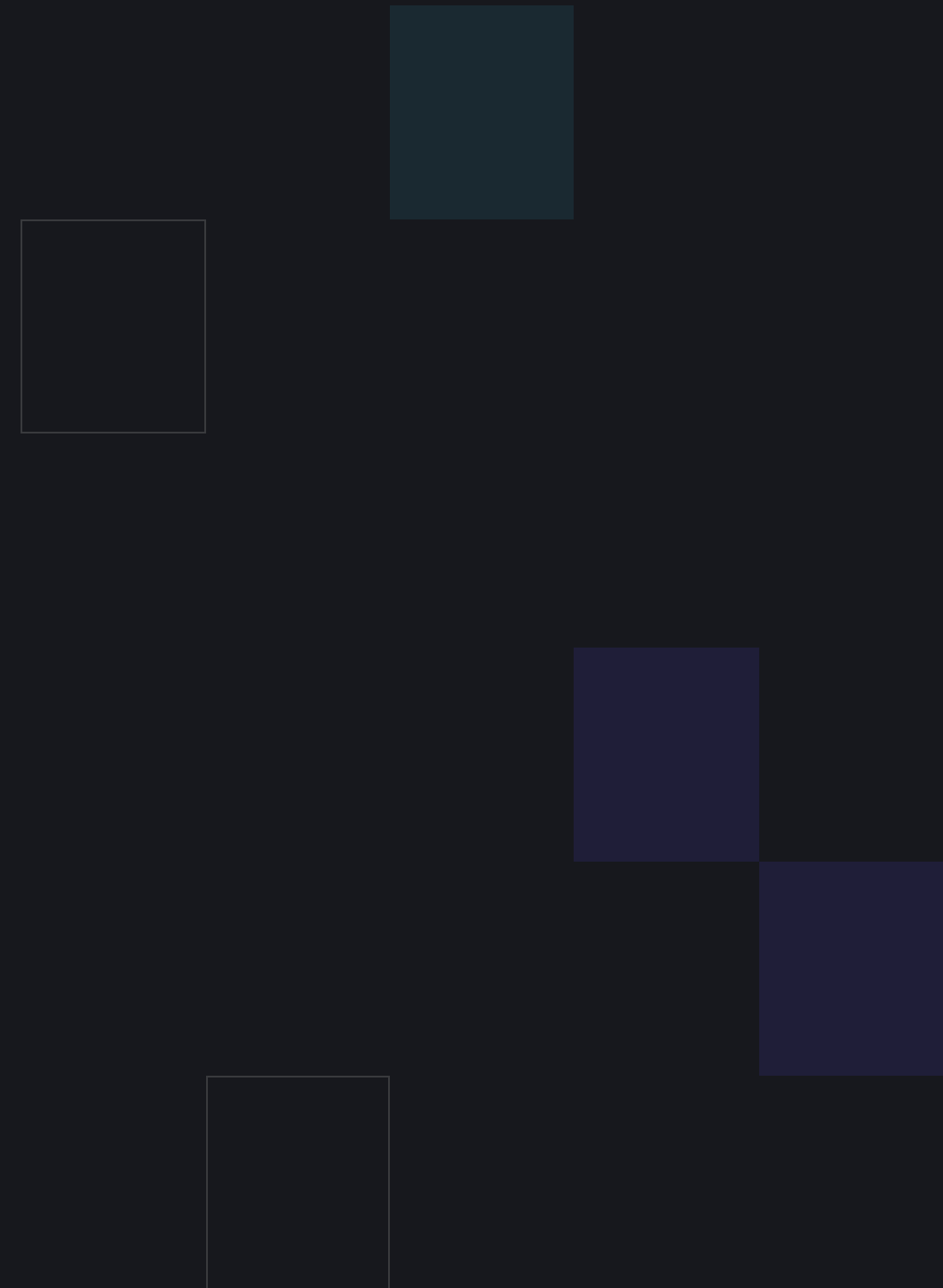
MODEL	BACKBONE SHAPE	PRETRAINING	PRIMARY USE
ViT / DeiT	Isotropic	Supervised / distilled	Classification, transfer
Swin / PVT / MViT	Hierarchical	Supervised	Detection, segmentation, video
MAE / BEiT	Isotropic	Masked image modeling	Label-free transfer
DINOv2	Isotropic (+ registers)	Self-distillation	Frozen general features
CLIP / SigLIP	Dual encoder	Vision–language contrastive	Open-vocabulary, zero-shot
ViViT / TimeSformer	Factorized space-time	Supervised / SSL	Action, video reasoning

Read any model on three axes — backbone shape · pretraining objective · downstream interface. The name matters less than the coordinates.

PART III

Applications in robotics & vision

From dense prediction to open-vocabulary perception to vision–language–action policies — the same backbones, new heads.



Dense prediction: SegFormer & Mask2Former

SEGFORMER

Efficient hierarchical encoder + MLP head

- ▶ Positional-encoding-free; conv in the FFN leaks in location.
- ▶ All-MLP decoder fuses multi-scale features — very lightweight.
- ▶ Strong accuracy/latency trade — apt for real-time robotics.

MASK2FORMER

One model for all segmentation

- ▶ Learnable queries each predict one mask — set prediction.
- ▶ Masked attention confines cross-attention to a query's region.
- ▶ Unifies semantic · instance · panoptic in one architecture.

Paradigm shift — segmentation becomes "predict a set of masks with attention," collapsing three task-specific architectures into one.

Open-vocabulary detection & segmentation: DETR & SAM

DETR · DETECTION AS SET PREDICTION

- Learnable object queries emit all boxes at once — **no anchors, no NMS.**
- Bipartite (Hungarian) matching gives a unique target per query.
- Deformable DETR adds sparse attention → fast convergence.

SAM · PROMPTABLE SEGMENTATION

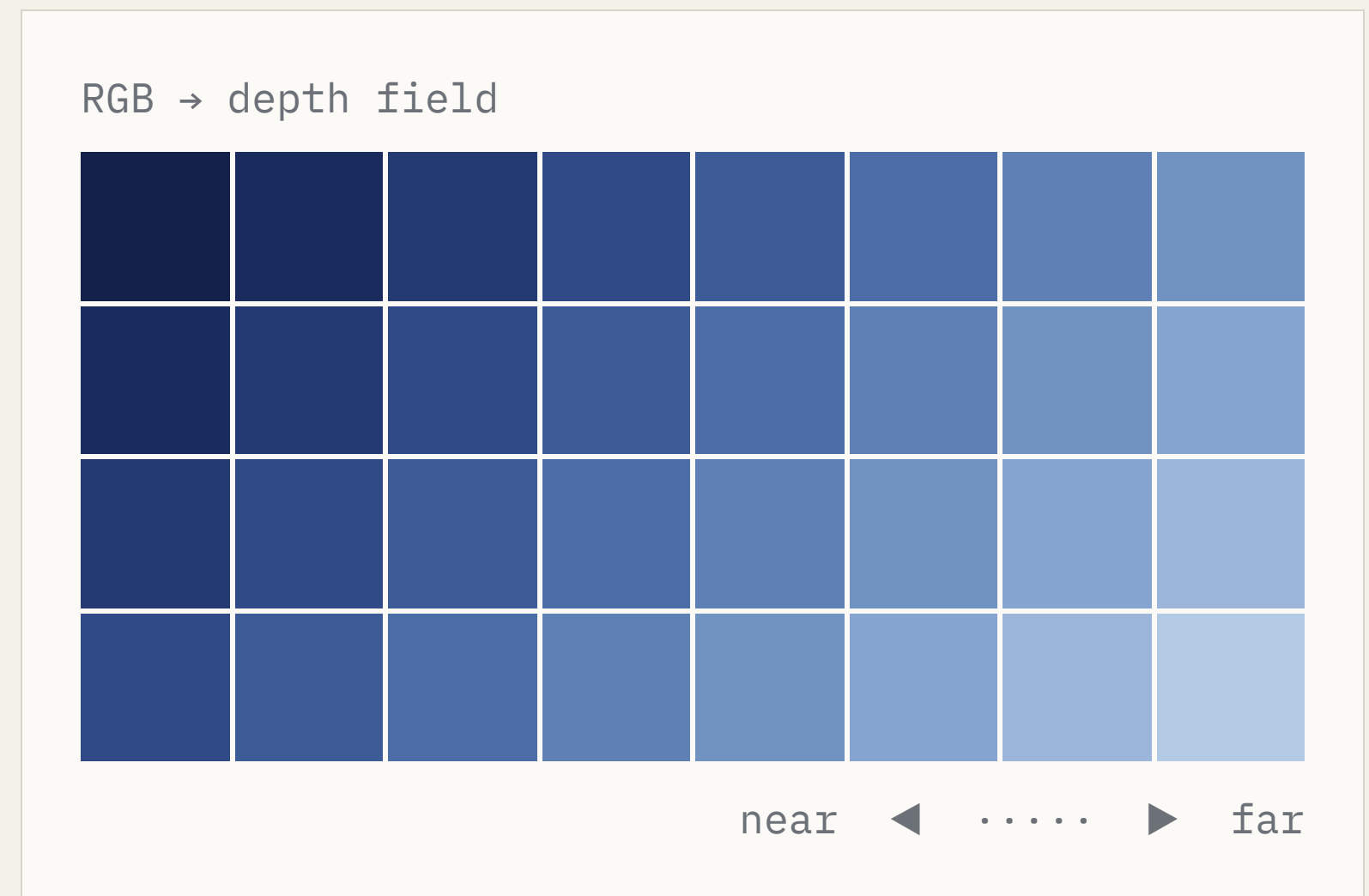
- Heavy ViT encoder once per image; a light decoder per prompt.
- Point / box / mask prompt → mask in milliseconds; trained on 1B masks.
- ★ **For manipulation** — class-agnostic masks on novel objects.

prompt → attention decoder → mask / box set

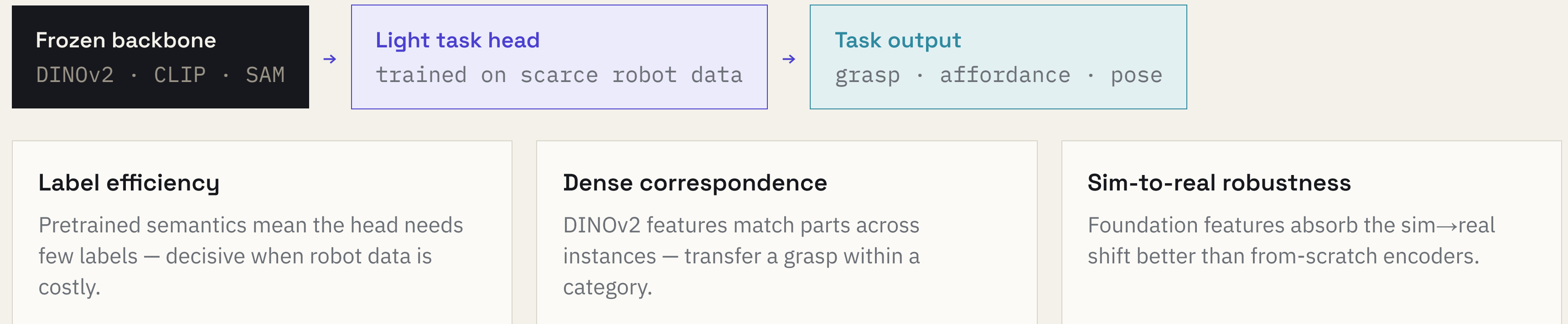
unifying view: queries decoded by attention

Monocular geometry: DPT & Depth Anything

- **DPT.** Reassemble tokens from several encoder stages into multi-resolution maps, then fuse — global coherence convs lack.
- **Consistent scale.** Attention's global field yields depth without smearing across object boundaries.
- **Depth Anything.** Tens of millions of labeled + pseudo-labeled images → strong zero-shot depth across unseen domains.
- ★ **For robots.** One forward pass → obstacle geometry for collision avoidance and grasp planning.

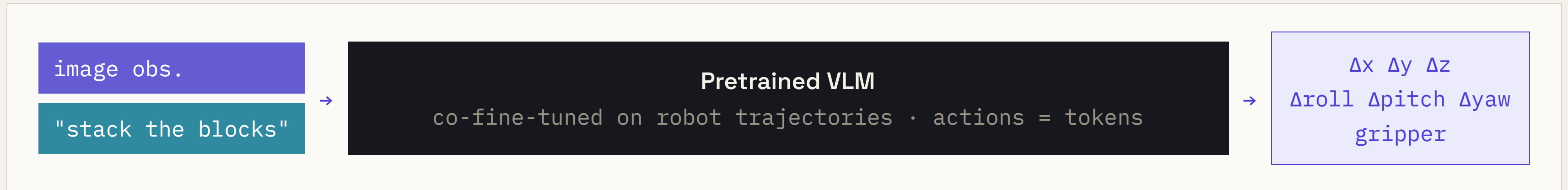


Foundation features as a robotic perception backbone



Honest caveat — frozen features can miss task-specific detail; light adaptation (LoRA, a few unfrozen blocks) often pays off — see Part IV.

Vision-language-action models: RT-2 & OpenVLA



- **RT-2.** Actions as discrete tokens in the text vocabulary → control is next-token prediction.
- **Emergent generalization.** Web-scale knowledge → follows instructions about objects unseen in robot data.
- **OpenVLA.** Open 7B-class VLA on a DINOv2 + SigLIP vision stack — reproducible baseline.
- ! **Open problems.** Inference latency, data scale, and reliability remain active research.

Spatio-temporal modeling for navigation & manipulation

SEQUENCE-MODELING POLICIES

Control as conditional sequence prediction

- ▶ Decision / Trajectory Transformer: predict the next action from (state, action, return) history.
- ▶ Long-horizon attention replaces bootstrapped value estimation.
- ▶ Attention over observation history → memory under partial observability.

PERCEPTION FOR DRIVING

BEV & occupancy transformers

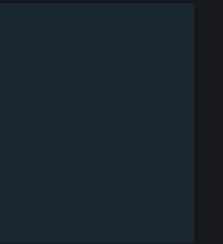
- ▶ BEVFormer attends across cameras and time → unified top-down map.
- ▶ Occupancy transformers lift this to dense 3D for planning.
- ▶ Multi-view + temporal fusion, all by attention.

The unifying idea — once perception is sequence modeling, fusing across views, time, and action is just more tokens and more attention: one architecture from sensing to control.

PART IV

Programming & implementation

PyTorch · timm · Hugging Face — instantiation, custom-dataset fine-tuning, mixed precision, profiling, and PEFT.



The ecosystem: PyTorch, timm, Hugging Face

SUBSTRATE

PyTorch

Autograd, `nn.Module`, AMP, the profiler, `torch.compile`. Everything else builds on it.

VISION BACKBONES

timm

Hundreds of ViT/hybrid variants + pretrained weights via `create_model`. Bring your own loop.

MODELS + TRAINER

HF transformers

Processors, configs, `Trainer`, the Hub — and multimodal models like CLIP/SigLIP.

FlashAttention · xFormers

fused attention

accelerate · DeepSpeed

multi-GPU & sharding

Model instantiation & inference

```
import torch
from transformers import (AutoModelForImageClassification,
                          AutoImageProcessor)

ckpt = "google/vit-base-patch16-224"
proc = AutoImageProcessor.from_pretrained(ckpt)
model = (AutoModelForImageClassification
         .from_pretrained(ckpt).eval().cuda())

# timm alternative – backbone only# model = timm.create_model("# vit_base_patch16_224")

batch = proc(img, return_tensors="pt").to("cuda")
with torch.inference_mode():
    logits = model(**batch).logits
pred = logits.argmax(-1).item()
print(model.config.id2label[pred])
```

Match the preprocessing

The processor applies the checkpoint's exact resize & normalization. Wrong stats silently destroy accuracy.

eval() + inference_mode()

Fixes dropout and skips autograd bookkeeping — faster than no_grad.

timm vs. transformers

timm for a raw backbone + custom loop; transformers for processors, Trainer, VLMs.

Patch embedding from scratch

```
class PatchEmbed(nn.Module):
    def __init__(self, img=224, patch=16, ch=3, dim=768):
        super().__init__()
        n = (img // patch) ** 2
        # tiling + projection in one conv
        self.proj = nn.Conv2d(ch, dim, patch, stride=patch)
        self.cls = nn.Parameter(torch.zeros(1, 1, dim))
        self.pos = nn.Parameter(torch.zeros(1, n+1, dim))

    def forward(self, x):
        # (B,3,H,W)
        x = self.proj(x) # (B,dim,H/16,W/16)
        x = x.flatten(2).transpose(1, 2) # (B,N,dim)
        cls = self.cls.expand(x.size(0), -1, -1)
        x = torch.cat([cls, x], dim=1) # prepend CLS
        return x + self.pos # add positions
```

The conv IS the tokenizer

kernel = stride = patch → non-overlapping tiling and linear projection in one op.

Positions are parameters

pos is shape (1, N+1, dim). For new resolutions, interpolate it.

Three lines of reshaping

Everything after is standard encoder blocks — this is the whole input stage.

Fine-tuning a custom dataset: data & transforms

```
from torchvision import transforms as T

MEAN, STD = (0.5,)*3, (0.5,)*3  # match ckpt!

train_tf = T.Compose([
    T.RandomResizedCrop(224, scale=(0.6, 1.0)),
    T.RandomHorizontalFlip(),
    T.RandAugment(num_ops=2, magnitude=9),
    T.ToTensor(), T.Normalize(MEAN, STD),
    T.RandomErasing(p=0.25),
])
eval_tf = T.Compose([
    T.Resize(256), T.CenterCrop(224),
    T.ToTensor(), T.Normalize(MEAN, STD),
])

train_dl = DataLoader(train_ds, batch_size=128,
                      shuffle=True, num_workers=8, pin_memory=True)
```

Normalization must match

Read MEAN/STD from the checkpoint's config — mismatched stats quietly cost accuracy.

Augmentation = data

The DeiT recipe. Add Mixup/CutMix in the collate fn for the strongest regime.

Keep the GPU fed

pin_memory + workers + non-blocking copies — loading is a silent bottleneck.

The training loop with mixed precision

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()
opt = torch.optim.AdamW(model.parameters(),
                        lr=5e-4, weight_decay=0.05)
sched = torch.optim.lr_scheduler.OneCycleLR(
    opt, max_lr=5e-4, total_steps=steps)

for x, y in train_dl:
    x = x.cuda(non_blocking=True)
    y = y.cuda(non_blocking=True)
    opt.zero_grad(set_to_none=True)
    with autocast(dtype=torch.float16):    # AMP
        loss = criterion(model(x), y)
    scaler.scale(loss).backward()
    scaler.unscale_(opt)
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(opt); scaler.update(); sched.step()
```

Why AMP

~½ the memory, often ~2× throughput on tensor cores; fp32 master weights stay stable.

fp16 needs the scaler

It rescales the loss so tiny gradients don't underflow. **bf16 can drop it.**

Clip at norm 1.0

ViTs are prone to loss spikes; clipping plus warmup tames them.

Optimization recipe: schedules, decay, regularization

```
# layer-wise LR decay – early layers move less
def param_groups(model, base, decay=0.01):
    groups = []
    for name, p in model.named_parameters():
        if not p.requires_grad: continue
        i = layer_id(name, L)          # 0 = stem
        scale = decay ** (L - i)
        wd = 0.0 if p.ndim <= 1 else 0.05
        groups.append(dict(params=p,
                           lr=base*scale, weight_decay=wd))
    return groups

opt = torch.optim.AdamW(
    param_groups(model, 5e-4), betas=(0.9, 0.999))
```

Cosine + warmup

Short linear warmup, then cosine decay — avoids early divergence.

No decay on norms/bias

Weight decay on matrices only; exempt 1-D params.

Label smoothing + drop-path

$\epsilon \approx 0.1$ and stochastic depth scaled to model size.

Weight EMA

Evaluate an EMA copy — a near-free accuracy bump.

Taming computational overhead

FlashAttention / SDPA	memory ↓, speed ↑ — exact
Grad checkpointing	~2× batch, +25% time
Grad accumulation	large batch, small VRAM
torch.compile	kernel fusion, throughput ↑
Token merging (ToMe)	attacks N in N ²

```
# 1 · fused attention (auto-selects Flash)
F.scaled_dot_product_attention(q, k, v)

# 2 · recompute activations, save memory
model.gradient_checkpointing_enable()

# 3 · graph capture + kernel fusion
model = torch.compile(model,
                       mode="max-autotune")

# 4 · simulate a 4× larger batch
loss = criterion(model(x), y) / accum
scaler.scale(loss).backward()
if (step + 1) % accum == 0:
    scaler.step(opt); scaler.update()
    opt.zero_grad(set_to_none=True)
```

Stack them freely — these levers are orthogonal. But profile first: guessing the bottleneck is the classic mistake.

Profiling & benchmarking

```
from torch.profiler import (profile,
                             ProfilerActivity as Act, schedule)

with profile(
    activities=[Act.CPU, Act.CUDA],
    schedule=schedule(wait=1, warmup=2, active=3),
    profile_memory=True, record_shapes=True,
) as prof:
    for step, (x, y) in enumerate(train_dl):
        train_step(x, y); prof.step()
        if step >= 6: breakpoint(prof.key_averages().table(
            sort_by="cuda_time_total", row_limit=10))
peak_gb = torch.cuda.max_memory_allocated() / 1e9
```

Warm up first

Early steps include autotuning & compile capture — the wait/warmup phases exclude them.

Two headline metrics

images/sec (cost) and **peak GB** (will it fit).

Read the signature

GPU gaps → data-bound. SDPA dominates → attention-bound. Match the lever, re-profile.

Parameter-efficient fine-tuning: LoRA

```
from peft import LoraConfig, get_peft_model

cfg = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=["query", "value"],
    bias="none",
)
model = get_peft_model(backbone, cfg)
model.print_trainable_parameters()

# trainable: 0.6M || total: 86.6M || 0.7%# one frozen backbone, many task adapters
model.save_pretrained("adapters/grasp")
# merge for zero-overhead inference:
merged = model.merge_and_unload()
```

TRAINABLE PARAMETERS

 **0.7%**

Rank-16 adapters on Q/V — full backbone frozen.

Tiny checkpoints, many tasks

One shared backbone + swappable per-skill adapters.

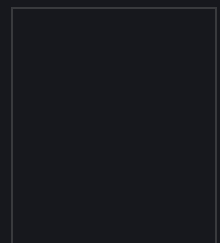
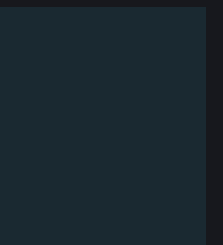
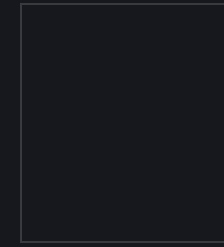
Siblings

Adapters, prompt tuning, QLoRA (4-bit base) for large models on one GPU.

PART V

Synthesis & practice

Choosing a backbone and an adaptation regime — and what to carry forward.



A practical decision guide

IF YOU NEED...	REACH FOR	ADAPTATION
Classification, abundant labels	ViT / Swin	Full fine-tune
Perception, scarce labels	DINOv2 (frozen)	Linear probe / LoRA
Segmentation / detection	Swin + Mask2Former	Fine-tune head + backbone
Open-vocabulary recognition	CLIP / SigLIP · SAM	Zero-shot / prompt
Monocular geometry	DPT / Depth Anything	Zero-shot / fine-tune
Language-conditioned manipulation	OpenVLA / RT-2	Co-fine-tune on demos

Orthogonal axis — compute budget. Tighter budgets push toward frozen features, PEFT, smaller patches over higher resolution, and FlashAttention.

Key takeaways

- 01 A ViT is an **image as a token sequence** through a transformer encoder — every variant perturbs one stage of that pipeline.

- 02 Attention buys **global context at quadratic cost** — that single tension explains the whole architecture landscape.

- 03 Self-supervised and vision–language pretraining turned ViTs into **foundation models** — frozen features are the default start.

- 04 In robotics, one backbone family serves perception, geometry, and control — converging on **vision–language–action** policies.

- 05 Rarely train from scratch — **pick a backbone, adapt lightly**: mixed precision, profiling, and PEFT are standard tooling.

References & further reading

FOUNDATIONS

Vaswani et al. — *Attention Is All You Need* (2017)

Dosovitskiy et al. — *An Image is Worth 16×16 Words* (ViT, 2021)

Touvron et al. — *Training data-efficient ViTs* (DeiT, 2021)

APPLICATIONS

Carion et al. — *DETR* (2020); Kirillov et al. — *Segment Anything* (2023)

Xie et al. — *SegFormer*; Cheng et al. — *Mask2Former* (2022)

Ranftl et al. — *DPT*; Yang et al. — *Depth Anything* (2024)

Brohan et al. — *RT-2* (2023); Kim et al. — *OpenVLA* (2024)

ARCHITECTURES

Liu et al. — *Swin Transformer* (2021); Wang et al. — *PVT* (2021)

He et al. — *Masked Autoencoders* (MAE, 2022); Bao et al. — *BEiT*

Oquab et al. — *DINOv2* (2023); Radford et al. — *CLIP* (2021)

Arnab et al. — *ViViT* (2021); Bertasius et al. — *TimeSformer*

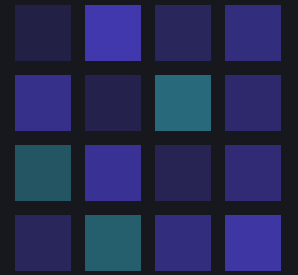
IMPLEMENTATION

Dao et al. — *FlashAttention 1/2* (2022–23)

Hu et al. — *LoRA* (2021); Dettmers et al. — *QLoRA* (2023)

Docs — `timm` · Hugging Face transformers / `peft`

PyTorch — AMP, `torch.profiler`, `torch.compile` guides



Thank you.

Vision is now sequence modeling — the skill that endures is choosing and adapting foundation backbones. Questions welcome.